

0

12

25

37

50

62

75

MANUAL TÉCNICO · V1.5.0

Sonda

Manual técnico do projeto

Motor de busca de texto completo escrito do zero em C# e .NET 10, com índice invertido em disco, ranking BM25, painel web e integrações abertas.

CARTA DO PROJETO

PROJETO	Sonda · busca de texto completo
DOCUMENTO	Manual técnico
VERSÃO	1.5.0
DATA	Outubro de 2026
AUTOR	Erik L. Pimenta
LICENÇA	MIT

14_2

31

Sumário

01 **Visão geral** 4

Para que serve 4

O que a Sonda faz 4

Novidades da 1.4.0 4

Ferramentas usadas 5

Identidade visual 5

02 **Conceitos** 7

Índice invertido 7

Análise de texto 7

Stemming RSLP e Porter 7

Relevância: BM25 e TF-IDF 7

WAND e saltos 8

Documentos semelhantes 8

03 **Como rodar** 9

Pré-requisitos 9

Primeira execução no Windows 9

Primeira execução no Linux e no macOS 9

Com Docker 9

Sem os scripts 9

Parar e limpar 10

04 **Guia do painel** 11

Navegação e barra de estado 11

Busca 11

Sugestões, histórico e buscas salvas 12

Explicação da pontuação 13

Visualizar documento 13

Visão geral do índice 14

Fontes 15

Termos 16

Integrações 17

Paleta de comandos e atalhos 18

Tema escuro e celular 19

05 **Sintaxe de busca** 21

Operadores 21

Dicas 21

06 **Linha de comando** 22

Comandos 22

Exemplos 22

07 **API REST** 24

Rotas 24

Exemplos 25

08 **Integrações** 26

Navegador (OpenSearch) 26

Feed Atom 26

Planilhas e dados 26

Monitoramento (Prometheus) 26

Webhooks 27

Exemplos de código 28

09 **Configuração** 30

Opções 30

Variáveis do Docker 31

10 **Desempenho e ajuste** 32

Resultados 32

Como ajustar 32

11 **Testes, CI e automação** 33

Testes 33

Integração contínua 33

Atividade diária 33

12 **Segurança e privacidade** 35

13 **Solução de problemas** 36

14 **Novidades da 1.5.0** 37

CAPÍTULO 01

Visão geral

A Sonda é um motor de busca de texto completo no estilo Lucene e Elasticsearch, escrita do zero em C#. Ela varre pastas com arquivos `.txt`, `.md` e `.html`, extrai o texto, quebra em termos, reduz cada palavra ao radical e grava um índice invertido binário em disco. As buscas voltam ordenadas por relevância em poucos milissegundos, com trechos destacados e explicação da pontuação.

1.1 Para que serve

Para encontrar informação em grandes volumes de documentos sem depender de `LIKE '%texto%'` num banco SQL, que lê todas as linhas, não entende plural nem acento e devolve os resultados sem nenhuma ordem de relevância.

O repositório entrega a estrutura, a arquitetura, a documentação e a ferramenta pronta para rodar. Quem usa decide quais pastas indexar, em que máquina, com qual porta, idioma, memória e chave de acesso.

1.2 O que a Sonda faz

- **Análise de texto própria:** tokenização Unicode, minúsculas, remoção de acentos, stopwords em português e inglês, stemming RSLP (português) e Porter (inglês) e detecção automática do idioma de cada documento.
- **Índice invertido em disco:** segmentos imutáveis gravados com `FileStream` em formato binário próprio, com codificação delta, inteiros de tamanho variável, blocos de 128 empacotados em bits, tabelas de saltos, campos armazenados comprimidos com Brotli e checksum CRC32 em todos os arquivos.
- **Indexação paralela:** uma tarefa varre as pastas e vários consumidores da Task Parallel Library analisam os arquivos ao mesmo tempo, gravando num buffer compartilhado em `ConcurrentDictionary`.
- **Relevância:** BM25 (padrão) ou TF-IDF clássico, peso maior para o título, estatísticas globais entre segmentos e explicação detalhada de cada pontuação.
- **Consultas:** todos ou qualquer termo, frases exatas e com distância, prefixos, busca aproximada com distância de edição, exclusão, grupos, campos, pesos e filtros por tipo, idioma e fonte.
- **WAND:** nas buscas por qualquer termo, o algoritmo pula documentos que não têm como entrar no top 10.
- **Indexação incremental:** arquivos sem mudança são pulados, alterados são substituídos e removidos saem do índice. Pastas podem ser observadas e reindexadas sozinhas.
- **Manutenção automática:** mesclagem de segmentos em segundo plano, remoção definitiva de excluídos e verificação de integridade.
- **Painel web** em HTML, CSS e JavaScript puros, servido pelo próprio ASP.NET Core, com sugestões enquanto você digita, paleta de comandos, atalhos de teclado, tema claro e escuro e layout para celular. Também há API REST e ferramenta de linha de comando.
- **Integrações abertas:** exportação em CSV, JSON e NDJSON, feed Atom de qualquer busca, OpenSearch para o navegador, métricas no formato do Prometheus e webhooks assinados com HMAC.

1.3 Novidades da 1.4.0

Identidade própria
Painel redesenhado com a linguagem de uma carta náutica: papel quente, tinta escura, magenta como único destaque e pontuações mostradas como sondagens.

Sugestões instantâneas
Enquanto você digita, aparecem as buscas anteriores e os documentos que já casam com o texto, para abrir direto.

Histórico e buscas salvas
As últimas buscas ficam no navegador e qualquer busca pode ser fixada com **Salvar**.

Paleta de comandos e atalhos
`Ctrl` `K` abre buscas, documentos, telas e ações; atalhos de teclado cobrem a navegação inteira.

Documento mais útil
Navegação entre ocorrências, documentos semelhantes, copiar caminho e baixar o texto.

Consultas
A Visão geral mostra o que foi pesquisado, as buscas sem resultado, o tempo médio e o p95.

Exportação e feed
Qualquer busca vira CSV, JSON, NDJSON ou feed Atom, com os mesmos filtros do painel.

Integrações
Tela nova com OpenSearch, métricas do Prometheus, webhooks com assinatura e exemplos de código prontos.

Nenhuma novidade muda o formato do índice: uma pasta `dados/` criada por versões anteriores continua funcionando sem reindexar.

1.4 Ferramentas usadas

FERRAMENTA	VERSÃO	PAPEL NO PROJETO
C# e .NET	.NET 10	Linguagem e plataforma de todo o motor, do servidor, da linha de comando e dos testes
Task Parallel Library	.NET 10	`Parallel.ForEachAsync` e `Parallel.For` distribuem a análise de documentos entre os núcleos
System.Threading.Channels	.NET 10	Fila limitada entre o varredor de pastas e os consumidores, com contrapressão
ConcurrentDictionary	.NET 10	Buffer de termos em memória, cache de radicais, mapa de chaves e estatísticas
FileStream e RandomAccess	.NET 10	Gravação sequencial dos segmentos e leitura posicional sem travas pelos leitores
System.IO.Compression	.NET 10	Compressão dos blocos de campos armazenados (Brotli)
ASP.NET Core	.NET 10	API REST com Minimal APIs, eventos em tempo real (Server-Sent Events), feed, métricas, webhooks e hospedagem do painel
HTML, CSS e JavaScript	ES2022	Painel sem frameworks nem CDN, com as fontes Archivo e IBM Plex Mono incluídas no projeto
xUnit	2.9	Testes de análise, armazenamento, busca, concorrência, mesclagem, indexação de pastas e exportação
Docker Compose	v2	Imagem multiestágio com SDK para compilar e ASP.NET para rodar
GitHub Actions	—	Integração contínua e a automação de atividade diária

O motor não usa nenhum pacote NuGet externo: tokenizador, stemmers, extrator de HTML, codecs binários, CRC32, parser de consultas, pontuação, WAND, destaque de trechos, exportação e assinatura dos webhooks foram escritos do zero ou com a biblioteca padrão do .NET.

1.5 Identidade visual

A 1.4.0 troca a paleta genérica por uma linguagem própria, tirada das cartas náuticas e do levantamento batimétrico, que é o que uma sonda faz: medir profundidade. O fundo lembra papel de carta, os textos usam uma tinta azul-petróleo quase preta, as bordas são linhas finas de graticulado e os cantos são quase retos. Só existe um destaque, o magenta usado nas cartas para avisos e luzes: menu ativo, termos encontrados, foco do teclado e ações principais.

A pontuação de cada resultado aparece como uma **sondagem**: número em itálico monoespaçado, com os decimais menores, e uma régua de profundidade que compara o resultado com o primeiro da lista. Tons de água em três profundidades preenchem barras e distribuições. A marca, um prumo de sondagem descendo com duas ondas de sonar, continua sempre em preto e branco.



Em cada amostra, a metade esquerda é o tema claro e a direita o escuro (exceto nas faixas de água e nos estados, que mostram a escala).

FONTE	PESOS	USO
Archivo	variável 100–900	Interface, títulos e textos. Os títulos usam a largura condensada do eixo `wdth`
IBM Plex Mono	400, 500 e itálico	Consultas, caminhos, termos, números de pontuação, código e rótulos

As duas fontes estão no projeto, em `wwwroot/fonts`, sob a SIL Open Font License. Nada vem de CDN.

CAPÍTULO 02

Conceitos

O que acontece entre um arquivo na pasta e um resultado na tela.

2.1 Índice invertido

Um índice invertido guarda, para cada termo, a lista ordenada dos documentos em que ele aparece, com a frequência e as posições. Buscar “índice invertido” vira ler duas listas e cruzar as duas, em vez de ler todos os documentos.

A Sonda divide o índice em segmentos. Cada segmento é um índice completo e imutável. Documentos novos ficam num buffer em memória e viram um segmento quando o buffer enche ou quando há um commit. Exclusões ficam num mapa de bits à parte, e segmentos pequenos são mesclados em segundo plano.

2.2 Análise de texto

```
"Os Motores de BUSCA indexaram páginas"
| tokenização       Os | Motores | de | BUSCA | indexaram | páginas
| minúsculas        os | motores | de | busca | indexaram | páginas
| stopwords         – | motores | – | busca | indexaram | páginas  (posições mantidas)
| remoção de acentos  | motores |   | busca | indexaram | paginas
└ stemming (RSLP)     | motor   |   | busc  | index     | pagin
```

A mesma análise roda na indexação e na consulta, então quem procura “páginas indexadas” encontra o documento acima. As palavras descartadas continuam contando posição, para que as frases respeitem a distância original.

O idioma de cada documento é detectado pela proporção de palavras funcionais de cada língua (ou pelo atributo `lang` do HTML). No modo automático, cada palavra da consulta gera as duas variantes, português e inglês, combinadas como sinônimos.

2.3 Stemming RSLP e Porter

PALAVRA	RADICAL	REGRA APLICADA
casas, casa	`cas`	plural e vogal final
professora, professor	`profes`	feminino e depois sufixo de substantivo
canções, canção	`canca`	plural ões e aumentativo com exceção
indexação, indexado, indexar	`index`	sufixos de substantivo e verbo
rapidamente	`rapid`	sufixo de advérbio
searching, searches	`search`	Porter, passo 1
generalizations	`gener`	Porter, passos 1 a 4

O RSLP roda sobre o texto já sem acentos, com as regras convertidas da mesma forma. Assim “relevancia” e “relevância” chegam ao mesmo radical, o que importa muito para quem digita sem acento.

2.4 Relevância: BM25 e TF-IDF

```
BM25(t, d) = boost × idf(t) × freq × (k1 + 1) / (freq + k1 × (1 − b + b × dl / avgdl))
idf(t) = ln(1 + (N − df + 0,5) / (df + 0,5))
```

`freq` é quantas vezes o termo aparece no campo, `dl` o tamanho do campo, `avgdl` o tamanho médio, `N` os documentos com o campo e `df` os que contêm o termo. Os padrões são k1 = 1,2 e b = 0,75. O título recebe peso 2.

```
TF-IDF(t, d) = boost × √freq × idf(t)² × 1/√dl     idf(t) = 1 + ln((N + 1) / (df + 1))
```

O BM25 satura a frequência: a décima ocorrência vale bem menos que a primeira. No TF-IDF clássico, repetir uma palavra cem vezes rende pontos demais. Dá para alternar entre os dois no painel, na API e na linha de comando.

2.5 WAND e saltos

Cada termo guarda a maior pontuação possível que ele pode dar (calculada pela maior frequência e pelo menor campo da lista). Numa busca por qualquer termo, o algoritmo WAND ordena os cursores pelo documento atual e só avalia um documento quando a soma dos máximos dos termos que podem estar nele supera a menor pontuação do top 10. Documentos sem chance são pulados com as tabelas de saltos.

Em 100 mil documentos, uma busca por três termos avalia em média 756 documentos com WAND contra 30.809 na busca exaustiva. Quando o painel precisa da contagem exata e das facetas, a busca exaustiva é usada.

2.6 Documentos semelhantes

A busca “mais como este” da 1.4.0 lê o texto armazenado de um documento, passa pela mesma análise e escolhe os termos mais característicos dele: os que aparecem mais no documento e menos no resto da coleção. Esses termos viram uma busca por qualquer termo, com WAND, que exclui o próprio documento. O resultado mostra os termos usados, para que dê para entender por que dois documentos foram considerados parecidos.

CAPÍTULO 03

Como rodar

A Sonda roda com o .NET SDK 10 instalado ou com Docker. Os scripts de início escolhem sozinhos o que estiver disponível.

3.1 Pré-requisitos

- **.NET SDK 10** (recomendado): baixe em dot.net e confira com `dotnet --list-sdks`.
- **Ou Docker:** Docker Desktop no Windows e no macOS, Docker Engine com o plugin compose no Linux.
- **Git** para clonar o repositório.

3.2 Primeira execução no Windows

1. Clone o repositório: `git clone https://github.com/ravelroot/Inverted-Index-Search-Engine.git`.
2. Abra a pasta e dê dois cliques em `iniciar.cmd`.
3. O script cria o `.env` a partir do `.env.example` e gera uma chave de acesso aleatória.
4. Com o .NET SDK 10 instalado, ele compila e inicia o servidor; sem o SDK, sobe o Docker.
5. O navegador abre em **http://localhost:8080** com os 30 documentos de exemplo já indexados. A chave aparece no terminal.

3.3 Primeira execução no Linux e no macOS

1. Clone o repositório e entre na pasta.
2. Rode `./iniciar.sh`.
3. Acesse **http://localhost:8080**.

3.4 Com Docker

1. Copie `.env.example` para `.env`.
2. Em `SONDA_DOCUMENTOS`, coloque a pasta do seu computador que será indexada (somente leitura dentro do contêiner).
3. Preencha `SONDA_API_KEY` com uma chave longa e aleatória.
4. Rode `docker compose up -d --build` e acesse **http://localhost:8080**.

O índice fica no volume `sonda-dados`. A porta é presa a `127.0.0.1`, então só a própria máquina acessa.

3.5 Sem os scripts

```
$ dotnet build src/Sonda.Server -c Release
$ dotnet src/Sonda.Server/bin/Release/net10.0/Sonda.Server.dll
$ dotnet run --project src/Sonda.Cli -- indexar C:\Documentos --fonte documentos
$ dotnet run --project src/Sonda.Cli -- buscar "índice invertido"
```

Caminhos relativos do appsettings são resolvidos a partir da raiz do repositório (a pasta que tem o `Sonda.slnx`) ou da variável `SONDA_BASE`.

3.6 Parar e limpar

- **Servidor local:** feche a janela ou aperte `Ctrl` `C`. O buffer em memória é gravado antes de sair.
- **Docker:** `docker compose down` para parar e `docker compose down -v` para apagar o índice.
- Para recomeçar do zero sem Docker, apague a pasta `dados/`.

CAPÍTULO 04

Guia do painel

O painel tem cinco telas: Busca, Visão geral, Fontes, Termos e Integrações. Todas são alcançadas pelo menu superior, pela paleta de comandos (`Ctrl` `K`) ou por atalhos de teclado.

4.1 Navegação e barra de estado

O topo mostra a marca com a versão, o menu das telas, o botão **Comandos e documentos** (que abre a paleta), a lista de atalhos e a troca de tema. Logo abaixo, uma faixa fina de estado resume o índice em tempo real: documentos, segmentos, tamanho em disco, modelo de relevância, ponto de gravação atual e o indicador **ao vivo**, aceso enquanto a conexão de eventos (Server-Sent Events) está aberta. O rodapé traz atalhos para a API, a descrição OpenSearch e as métricas.

4.2 Busca

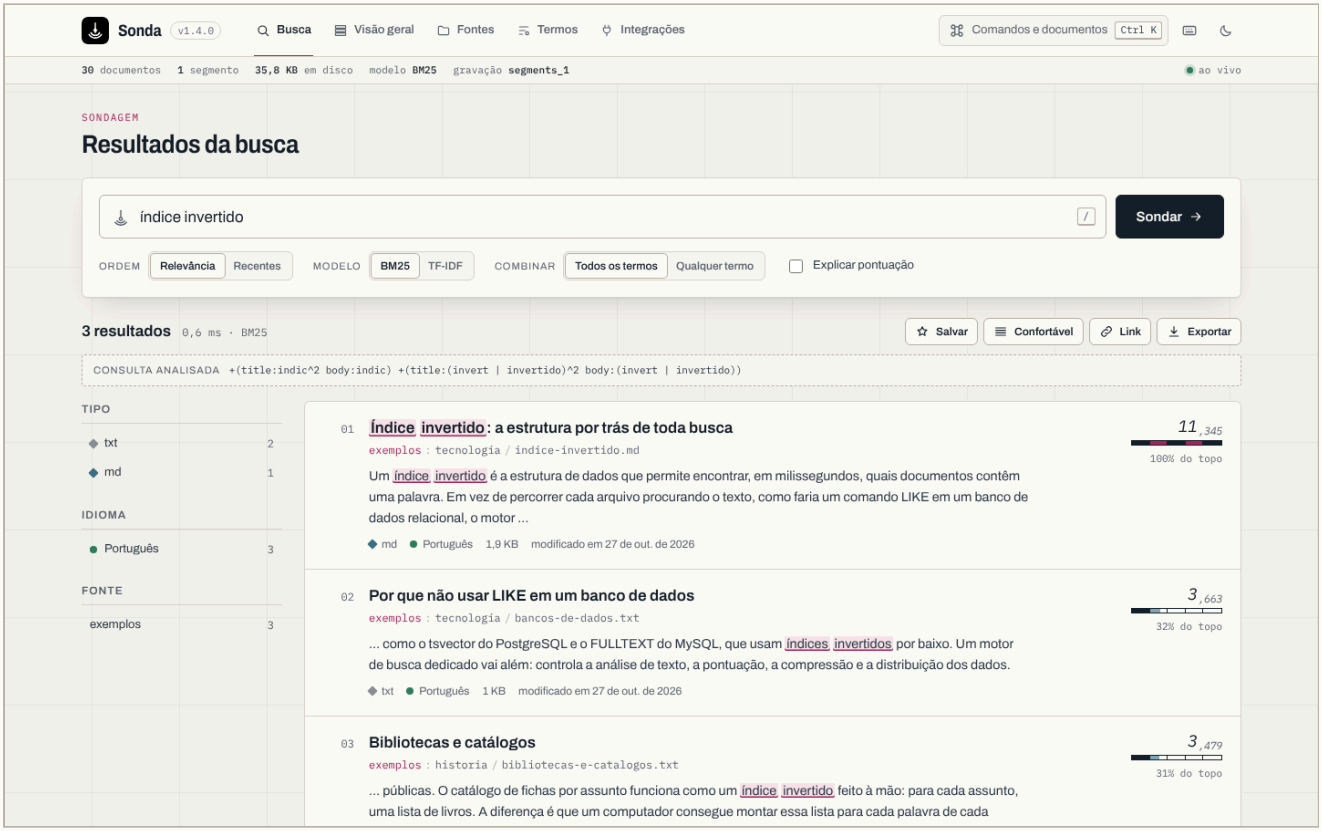


FIG. 4.1 Resultados para “índice invertido”: facetas à esquerda, títulos e trechos com os termos destacados e a pontuação de cada resultado como uma sondagem, à direita.

- **Sondar:** o botão (ou `Enter`) executa a busca. A tecla `/` leva o cursor para a caixa de qualquer ponto da tela.
- **Ordem:** por relevância ou pelos arquivos modificados mais recentemente.
- **Modelo:** BM25 ou TF-IDF, sem reindexar.
- **Combinar:** todos os termos ou qualquer termo. Se nenhum documento tiver todos, a busca é ampliada sozinha e um aviso aparece.

- **Consulta analisada:** mostra como a consulta ficou depois da análise: radicais, campos, pesos e operadores.
- **Facetas:** tipo, idioma e fonte com a contagem de resultados. Um clique filtra; outro clique remove o filtro.
- **Sondagem:** a pontuação aparece em itálico, com os decimais menores, e a régua abaixo dela mostra quanto o resultado vale em relação ao primeiro (“100% do topo”).
- **Barra de ações: Salvar** fixa a busca, **Confortável/Compacta** troca a densidade da lista, **Link** copia o endereço da busca com todos os filtros e **Exportar** baixa os resultados em CSV, JSON ou NDJSON, ou abre o feed Atom.
- **Teclado:** `J` e `K` descem e sobem pelos resultados; `Enter` ou `O` abre o documento selecionado.

4.3 Sugestões, histórico e buscas salvas

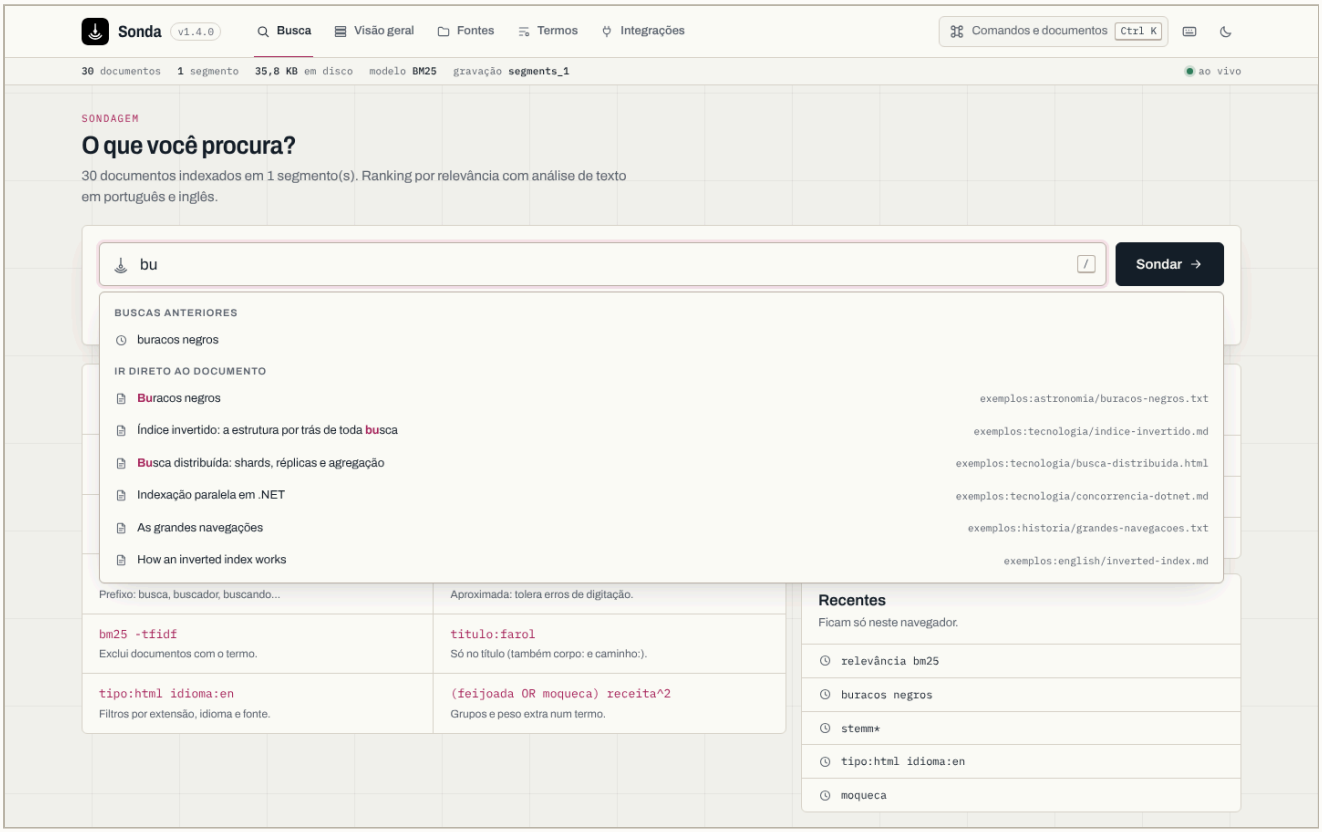


FIG. 4.2 Sugestões ao digitar “bu”: buscas anteriores e documentos cujo conteúdo já casa com o prefixo, prontos para abrir.

A partir da segunda letra, a caixa de busca mostra duas listas: **buscas anteriores** que começam com o que foi digitado e **ir direto ao documento**, com até seis documentos que casam com o texto, usando a última palavra como prefixo (`/api/sugestoes`). As setas escolhem, `Enter` confirma e `Esc` fecha.

Sem nenhuma consulta, a tela mostra o guia **Como pesquisar** com exemplos clicáveis, as buscas **recentes** e as **salvas**. Histórico e buscas salvas ficam só no navegador (armazenamento local): nada disso vai para o servidor, e cada pessoa vê apenas as suas.

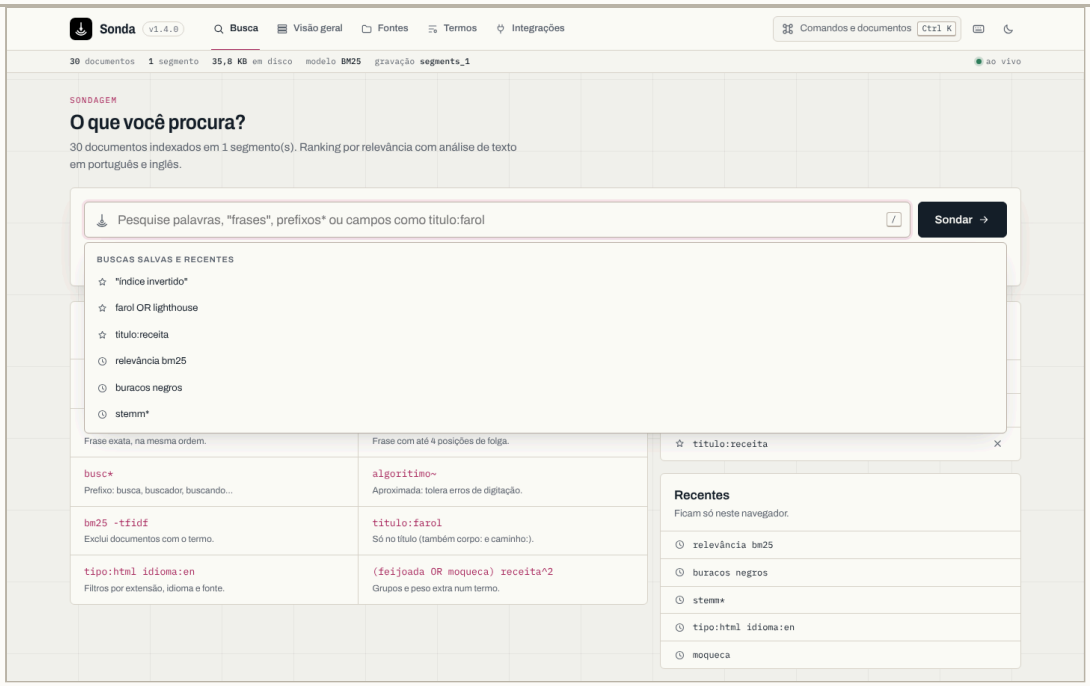


FIG. 4.3 Tela inicial da busca: guia de sintaxe, exemplos e buscas recentes.

4.4 Explicação da pontuação

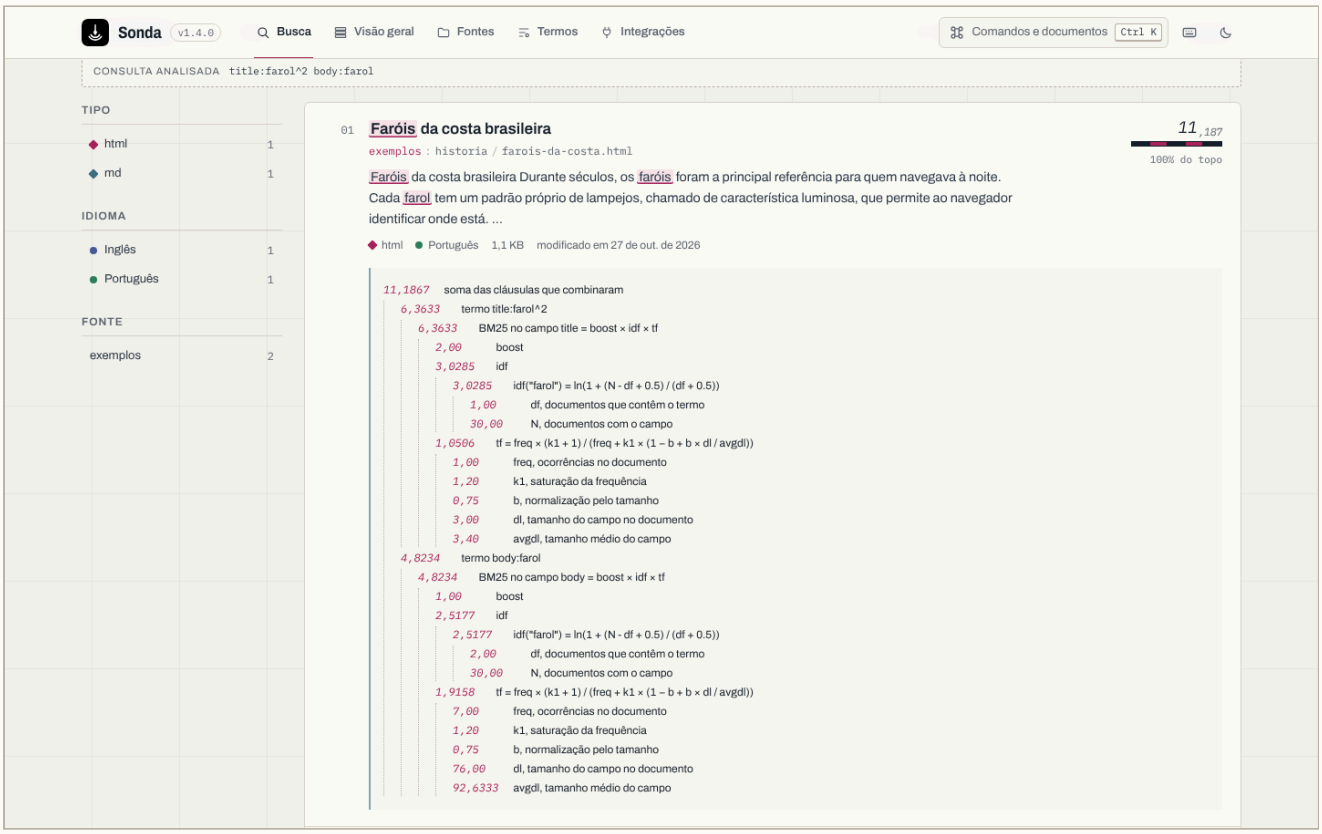


FIG. 4.4 Com **Explicar pontuação** marcado, cada resultado mostra a árvore de cálculo: boost, idf com df e N, e tf com frequência, k1, b, dl e avgdl.

A soma dos valores da árvore é exatamente a pontuação exibida no resultado.

4.5 Visualizar documento

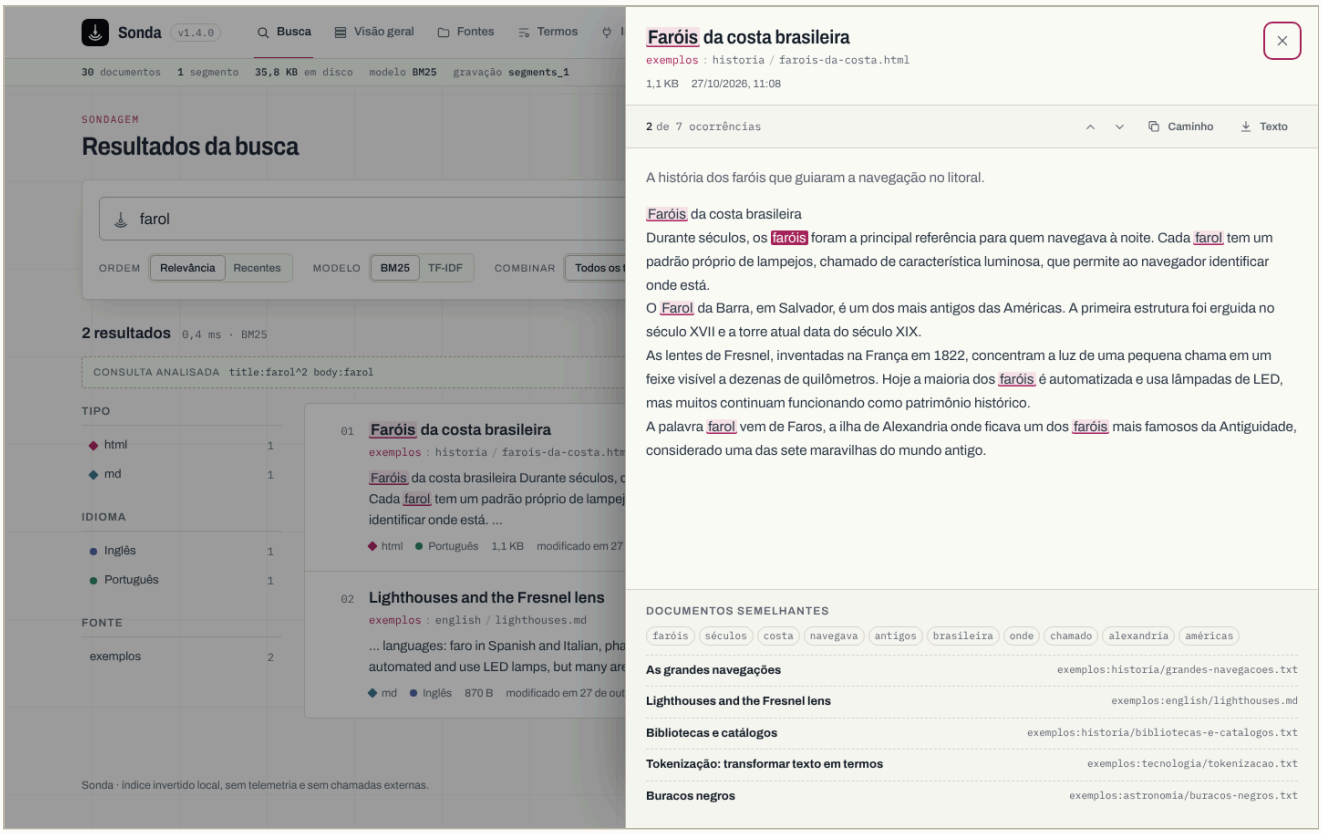


FIG. 4.5 Documento aberto na gaveta lateral, com as ocorrências destacadas, o contador “2 de 7 ocorrências” e os documentos semelhantes no fim.

- **Ocorrências:** as setas da barra (ou `N` e `Shift` `N`) pulam para a próxima e a anterior, rolando o texto até ela.
- **Caminho** (ou `C`): copia a chave `fonte:caminho` do documento.
- **Texto:** baixa o texto extraído como `.txt` (`/api/documentos/{chave}/texto`).
- **Documentos semelhantes:** os termos característicos do documento e os cinco documentos mais parecidos, que abrem na mesma gaveta.

`Esc` ou um clique fora da gaveta fecha a visualização. Documentos muito longos mostram os primeiros 60 mil caracteres.

4.6 Visão geral do índice

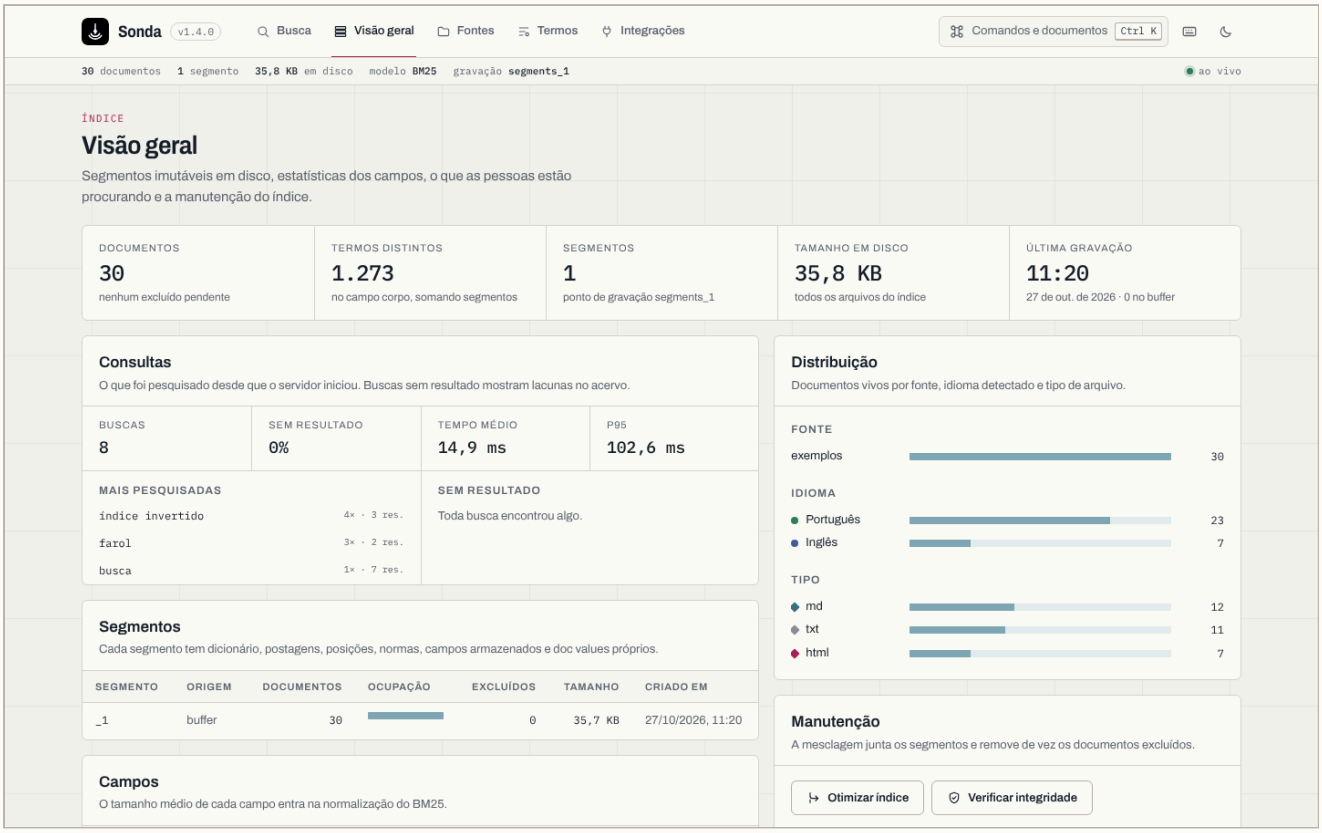


FIG. 4.6 Indicadores do índice, consultas feitas desde que o servidor iniciou, distribuição dos documentos, segmentos, campos e manutenção.

- **Indicadores:** documentos, termos distintos, segmentos, tamanho em disco e última gravação.
- **Consultas:** total de buscas, porcentagem sem resultado, tempo médio, p95, as mais pesquisadas e as que não acharam nada. As buscas sem resultado mostram lacunas no acervo ou palavras que precisam de sinônimos. Os números vêm de `/api/buscas/estatisticas` e zeram quando o servidor reinicia.
- **Distribuição:** documentos por fonte, idioma detectado e tipo de arquivo.
- **Segmentos:** nome, origem (buffer ou mesclagem), documentos, proporção de excluídos, tamanho e data.
- **Campos:** termos distintos, documentos, tokens e tamanho médio, usado no BM25.
- **Otimizar índice:** mescla todos os segmentos em um e remove de vez os excluídos.
- **Verificar integridade:** confere o CRC32 de todos os arquivos do ponto de gravação atual.

4.7 Fontes

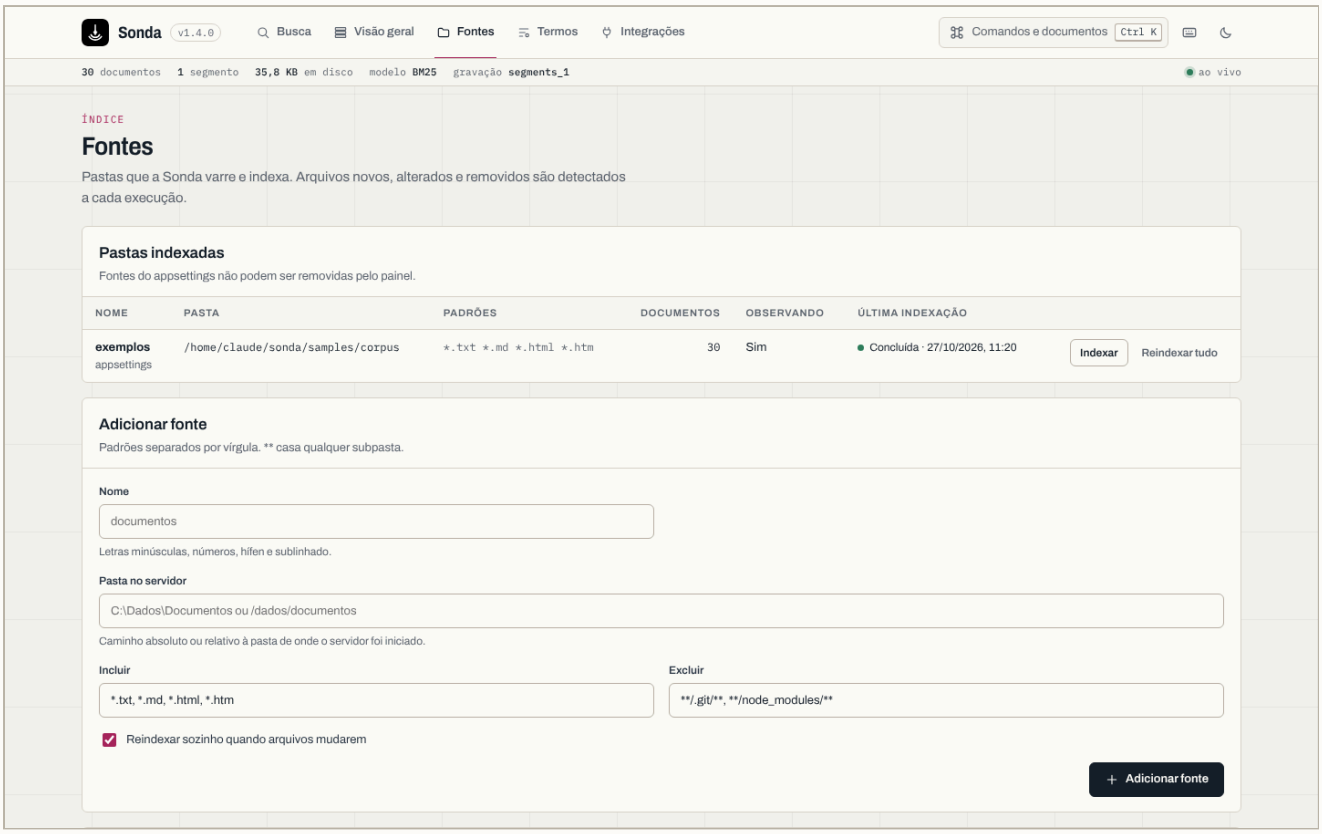


FIG. 4.7 Pastas indexadas, cadastro de novas fontes e histórico das execuções.

- **Indexar:** varredura incremental, que pula arquivos com a mesma data e tamanho.
- **Reindexar tudo:** reprocessa todos os arquivos da fonte.
- **Adicionar fonte:** nome, pasta no servidor, padrões de inclusão e exclusão e a opção de observar alterações.

Durante a indexação aparece o progresso em tempo real: arquivos encontrados, lidos, novos, atualizados, vazão e arquivo atual, com botão para cancelar. Ao terminar, a execução entra no histórico e, se houver webhooks configurados, o evento `execucao` é enviado.

Se o servidor tiver chave de acesso, o painel pede a chave antes de qualquer alteração. Ela fica só na memória da aba.

4.8 Termos

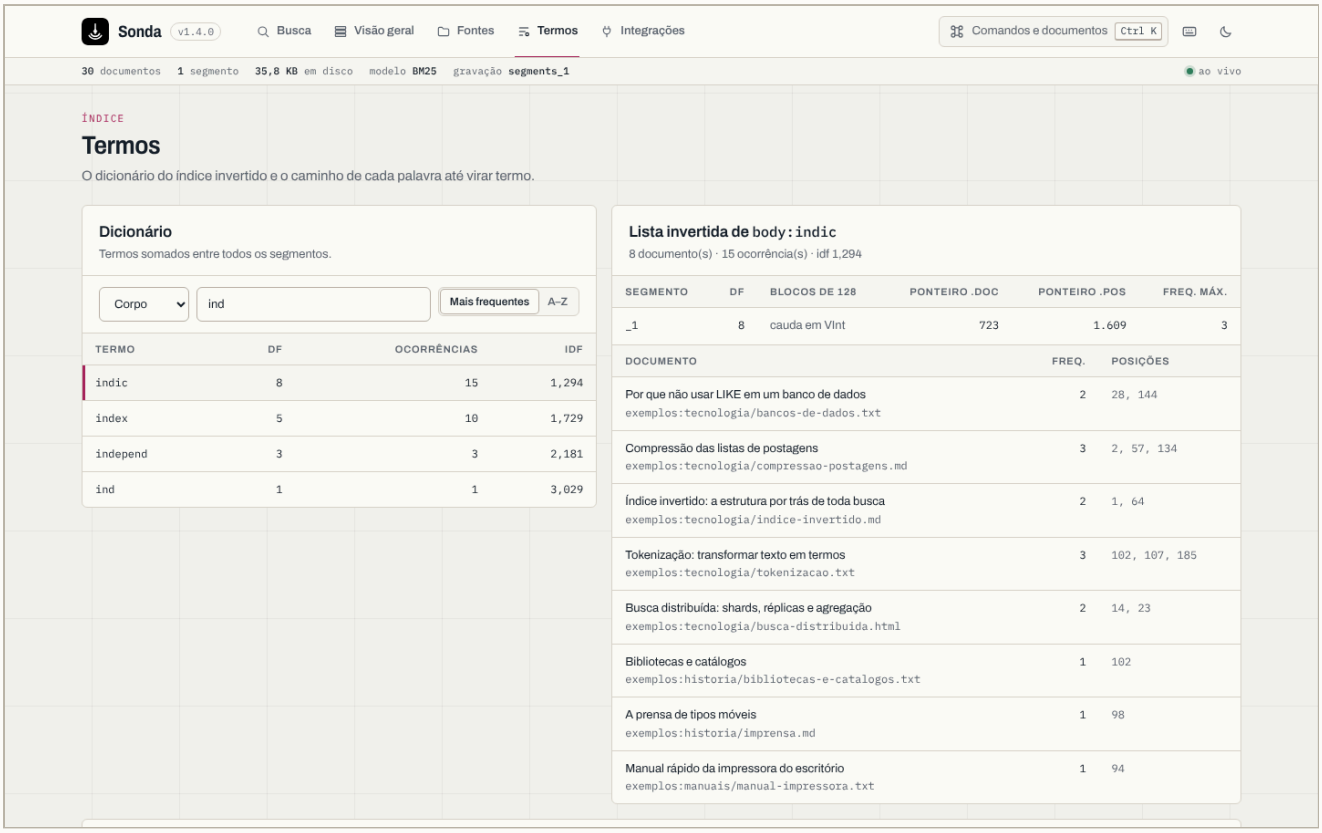


FIG. 4.8 O dicionário do campo corpo e a lista invertida de um termo.

- **Dicionário:** termos de um campo com df, ocorrências e idf, por frequência ou em ordem alfabética, com filtro por prefixo.
- **Lista invertida:** para o termo escolhido, os segmentos com ponteiros nos arquivos `.doc` e `.pos`, blocos de 128, frequência máxima, e os documentos com as posições de cada ocorrência.
- **Analisador:** mostra o caminho de cada palavra até virar termo, com os radicais em português e inglês e as stopwords descartadas.

4.9 Integrações

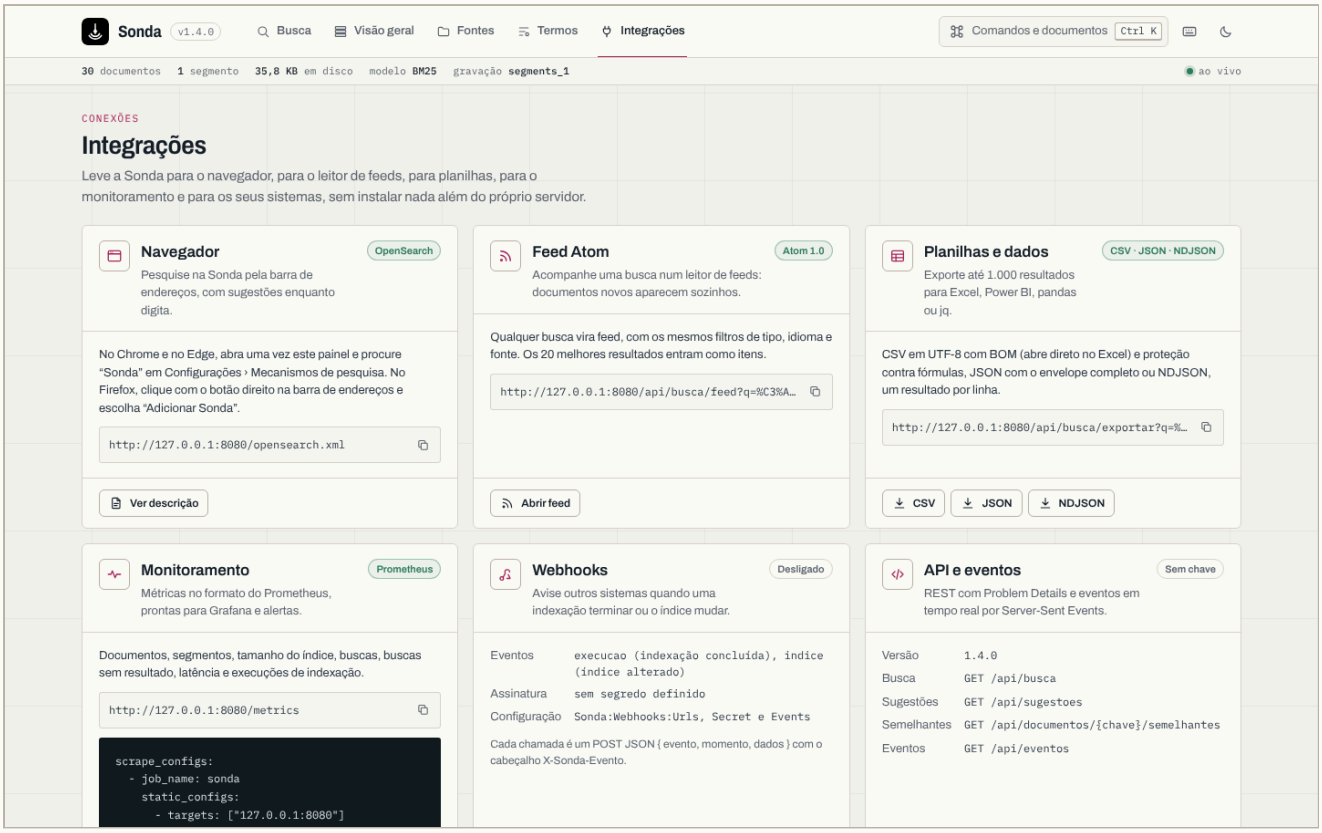


FIG. 4.9 Tela Integrações: navegador, feed Atom, planilhas e dados, monitoramento, webhooks e API, cada um com o endereço pronto para copiar.

A tela reúne tudo o que liga a Sonda a outras ferramentas, com o estado atual de cada item (por exemplo, webhooks ligados ou desligados e se a API exige chave). Cada cartão tem o endereço para copiar com um clique e um botão de ação: ver a descrição OpenSearch, abrir o feed, baixar CSV, JSON ou NDJSON. No fim, exemplos de código em curl, PowerShell, JavaScript e Python já montados com a última busca feita. O capítulo 8 detalha cada integração.

4.10 Paleta de comandos e atalhos

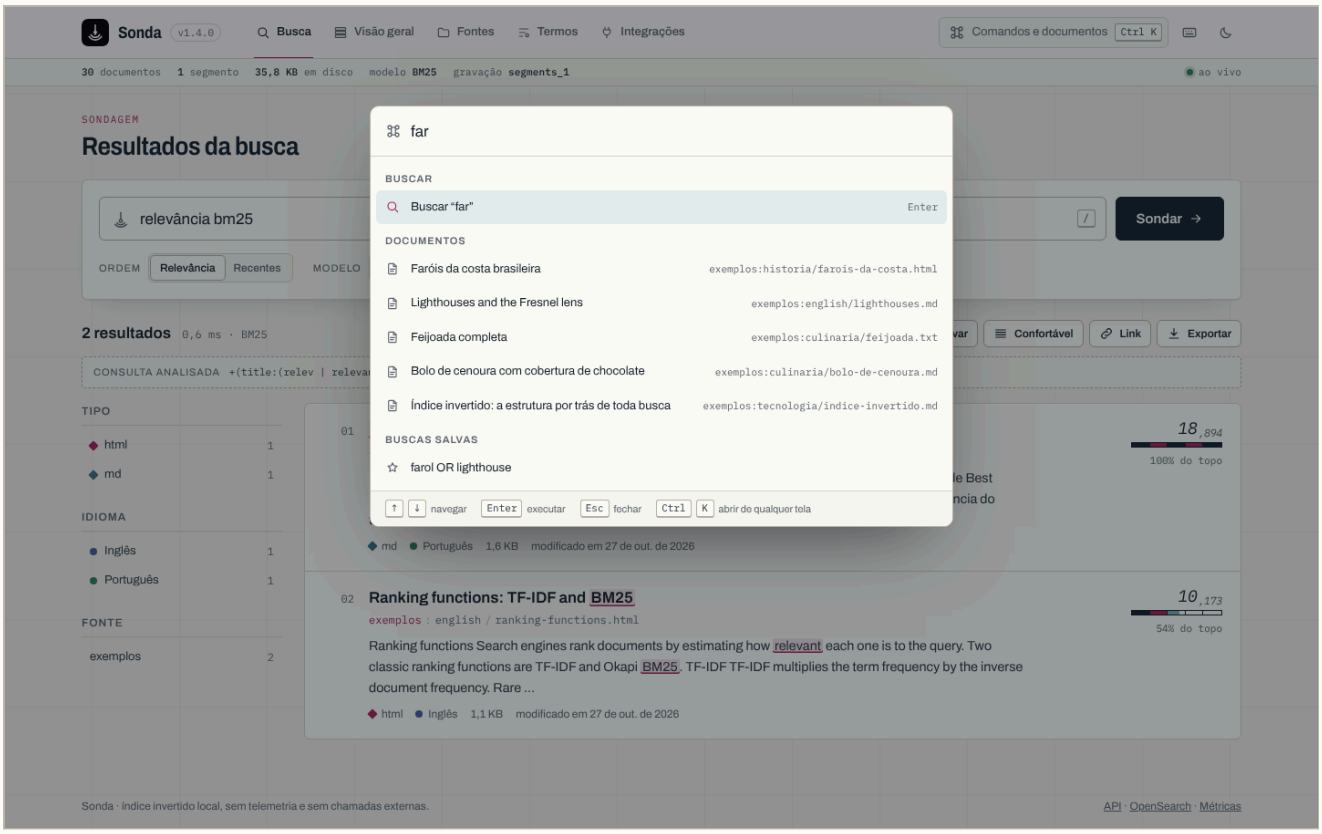


FIG. 4.10 Paleta de comandos com "far" digitado: buscar o texto, abrir um documento, uma busca salva, uma tela ou uma ação.

Ctrl K (ou ⌘ K no macOS) abre a paleta em qualquer tela. O que for digitado filtra ao mesmo tempo: a busca pelo texto, documentos que casam, buscas salvas e recentes, as telas e ações como alternar o tema, otimizar o índice e verificar a integridade.

ATALHO	O QUE FAZ
Ctrl K · ⌘ K	Abre a paleta de comandos
/	Leva o cursor para a caixa de busca
J · K	Próximo e anterior resultado
Enter · O	Abre o resultado selecionado
S	Salva a busca atual ou tira das salvas
N · Shift N	Próxima e anterior ocorrência no documento aberto
C	Copia o caminho do documento aberto
Esc	Fecha a paleta, as sugestões ou o documento
?	Mostra a lista de atalhos
G então B · I · F · T · N	Vai para Busca, Visão geral (índice), Fontes, Termos ou Integrações

Os atalhos de uma letra não disparam enquanto o cursor está numa caixa de texto.

4.11 Tema escuro e celular

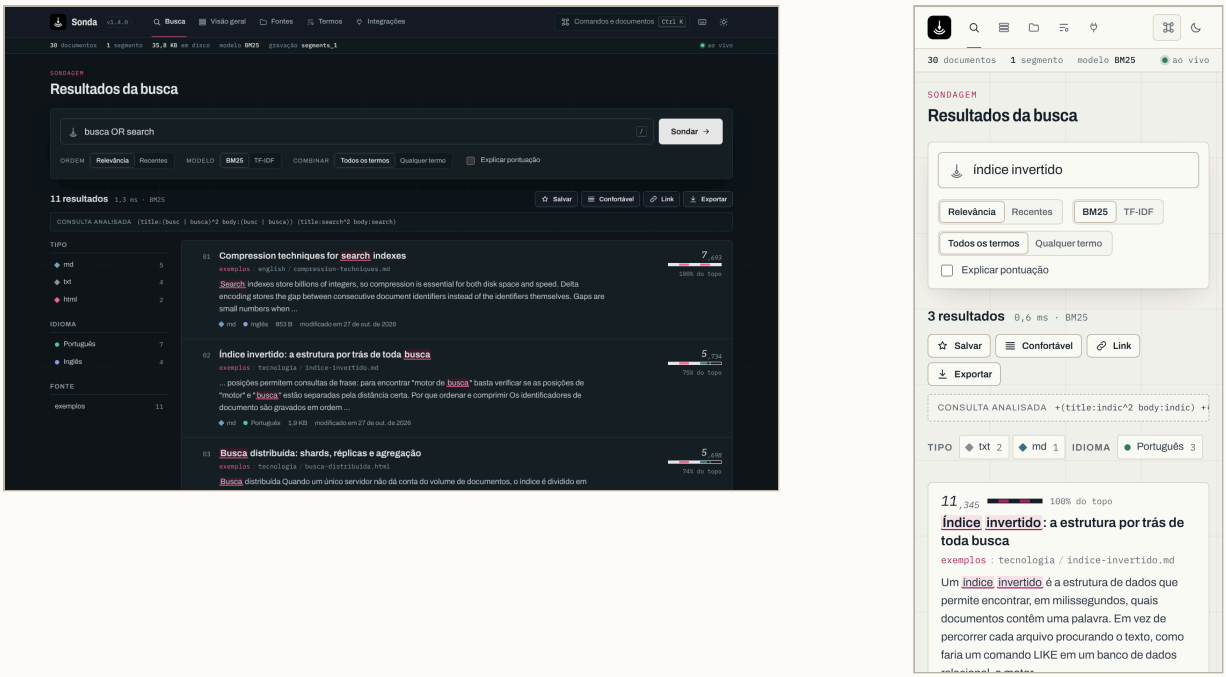


FIG. 4.11 Tema escuro, com as mesmas cores em tons de mar à noite, e o layout para celular.

O botão de lua no topo alterna o tema; a escolha fica guardada no navegador. Sem escolha, o painel segue o tema do sistema operacional. No celular o menu mostra só os ícones, as facetas ficam acima da lista e cada tela passa a ter uma coluna só.

CAPÍTULO 05

Sintaxe de busca

A sintaxe segue o Lucene, com operadores também em português. Os operadores precisam estar em maiúsculas.

5.1 Operadores

CONSULTA	O QUE FAZ
`motor busca`	Todos os termos (padrão). Com stemming, encontra “motores de buscas”
`motor OR busca` · `motor OU busca`	Qualquer um dos termos
`motor AND busca` · `motor E busca`	Todos os termos, mesmo com o operador padrão OU
`+motor busca`	`motor` obrigatório, `busca` opcional (aumenta a pontuação)
`bm25 -tfidf` · `bm25 NOT tfidf` · `NAO`	Exclui documentos com o termo
`"índice invertido"`	Frase exata, na ordem
`"motor relevância"~4`	Frase com até 4 posições de folga; ordem invertida custa mais
`busc*`	Prefixo; também casa radicais menores que o prefixo, como `busc` para `buscan*`
`algoritimo~` · `algoritimo~1`	Busca aproximada com até 2 (ou o número dado) edições, incluindo troca de letras vizinhas
`(feijoada OR moqueca) receita`	Grupos com parênteses
`farol^3 navio` · `(a b)^2`	Peso extra para um termo ou grupo
`titulo:farol` · `corpo:` · `caminho:`	Busca só no campo. Aceita `title`, `body` e `path`
`tipo:html` · `idioma:en` · `fonte:exemplos`	Filtros exatos, que não mudam a pontuação

5.2 Dicas

- Palavras muito comuns, como *de*, *para* e *the*, são ignoradas. Uma consulta só com elas não traz resultados.
- Acentos e maiúsculas não importam.
- Use frases quando a ordem das palavras importa e o operador OU quando qualquer termo serve.
- A consulta analisada mostra exatamente o que foi procurado. Ela é a melhor ferramenta para entender um resultado inesperado.
- Quando um documento é quase o que você queria, abra-o e veja **Documentos semelhantes**: costuma ser mais rápido do que reescrever a consulta.

CAPÍTULO 06

Linha de comando

A ferramenta `sonda` usa o mesmo motor do servidor. Comandos de leitura funcionam mesmo com o servidor aberto; os de escrita exigem que ele esteja parado, porque só um processo pode gravar no índice.

6.1 Comandos

COMANDO	O QUE FAZ
`indexar <pasta>`	Indexa a pasta. Opções: `--fonte`, `--incluir`, `--excluir`, `--completo`, `--paralelismo`, `--idioma auto|pt|en|nenhum`, `--ram`, `--max-mb`
`buscar "consulta"`	Busca. Opções: `--top`, `--pagina`, `--tfidf`, `--ou`, `--explicar`, `--recentes`, `--tipo`, `--fonte`, `--json`. Para exportar: `--formato csv|json|ndjson`, `--limite` (padrão 500) e `--saida <arquivo>`
`semelhantes <fonte:caminho>`	Documentos parecidos com um documento (mais como este). `--top`, `--json`
`estatisticas`	Documentos, segmentos, campos, tamanho e distribuição. `--json`
`termos`	Dicionário de um campo: `--campo`, `--prefixo`, `--limite`, `--frequencia`
`postagens <termo>`	Lista invertida de um termo, com ponteiros, blocos e posições
`analisar "texto"`	Tokenização, stopwords e radicais RSLP e Porter
`otimizar`	Mescla os segmentos (`--segmentos 1`) e remove excluídos
`verificar`	Confere o checksum CRC32 de todos os arquivos
`remover-fonte <nome>`	Remove do índice os documentos de uma fonte
`benchmark`	Gera um corpus sintético, indexa e mede consultas: `--docs`, `--consultas`, `--manter`

Opção comum a todos: `--indice <pasta>`, ou a variável `SONDA_INDICE`. O padrão é `dados/indice`.

6.2 Exemplos

```
$ sonda indexar samples/corpus --fonte exemplos
Encontrados  Novos  Atualizados  Sem mudança  Removidos  Falhas  Lidos    Tempo
         30     30            0            0          0       0  30,0 KB  0,17 s

$ sonda buscar "relevância bm25" --top 3
2 resultado(s) · BM25
Consulta interpretada: +(title:(relev | relevancia)^2 body:(relev | relevancia)) +(title:bm25^2
body:bm25)
 1. Relevância com BM25
    exemplos:tecnologia/bm25-relevancia.md · md · pt · 1,6 KB · pontuação 18,894
 2. Ranking functions: TF-IDF and BM25
    exemplos:english/ranking-functions.html · html · en · 1,1 KB · pontuação 10,173

$ sonda postagens indic --campo corpo --limite 3
body:indic · df 8 · ocorrências 15 · idf 1,294
Documento                                   Freq.  Posições
exemplos:tecnologia/bancos-de-dados.txt         2  28, 144
exemplos:tecnologia/compressao-postagens.md      3  2, 57, 134
exemplos:tecnologia/indice-invertido.md          2  1, 64
```

```
# exporta até 1.000 resultados para uma planilha
$ sonda buscar "farol OR lighthouse" --formato csv --limite 1000 --saida farois.csv

# NDJSON direto para o jq
$ sonda buscar "tipo:md" --formato ndjson | jq -r '.titulo'

# documentos parecidos com um documento
$ sonda semelhantes exemplos:tecnologia/bm25-relevancia.md --top 5
```

CAPÍTULO 07

API REST

As respostas são JSON, salvo exportações, feed e métricas. Com `Sonda:Api:ApiKey` definido, rotas que alteram o índice exigem o cabeçalho `X-Sonda-Key`; com `ProtectReads`, as de leitura também.

7.1 Rotas

MÉTODO	ROTA	FUNÇÃO
GET	`/api/busca?q=&pagina=&tamanho=&ordem=&similaridade=&operador=&explicar=&tipo=&idioma=&fonte=`	Busca com trechos, facetas e, opcionalmente, explicação
GET	`/api/busca/exportar?q=&formato=csv|json|ndjson&limite=`	NOVO. Resultados como anexo, com os mesmos filtros da busca
GET	`/api/busca/feed?q=`	NOVO. Feed Atom com os 20 primeiros resultados
GET	`/api/sugestoes?q=&limite=&formato=`	NOVO. Documentos que casam com o texto digitado; `formato=opensearch` para o navegador
GET	`/api/documentos/{chave}?q=`	Documento completo com destaques. A chave vai em Base64 URL
GET	`/api/documentos/{chave}/semelhantes?limite=`	NOVO. Termos característicos e documentos parecidos
GET	`/api/documentos/{chave}/texto`	NOVO. Texto extraído como `.txt`
GET	`/api/indice`	Estatísticas do índice, segmentos e campos
POST	`/api/indice/otimizar?segmentos=1`	Mesclagem forçada
POST	`/api/indice/verificar`	Verificação de checksums
GET	`/api/termos?campo=&prefixo=&limite=&ordem=`	Dicionário
GET	`/api/termos/postagens?campo=&termo=&limite=`	Lista invertida de um termo
GET	`/api/analise?texto=`	Análise de texto passo a passo
GET · POST	`/api/fontes`	Lista e cadastra fontes
DELETE	`/api/fontes/{nome}`	Remove uma fonte do catálogo e seus documentos
POST	`/api/fontes/{nome}/indexar?modo=incremental|completa`	Agenda uma indexação
GET · POST	`/api/indexacao` · `/api/indexacao/cancelar`	Estado da fila e cancelamento
GET	`/api/execucoes?limite=`	Histórico de indexação
GET	`/api/buscas/estatisticas`	NOVO. Consultas recentes, mais frequentes e sem resultado, média e p95
GET	`/api/integracoes`	NOVO. Endereços e estado das integrações
GET	`/api/eventos`	Server-Sent Events: `progresso`, `fila`, `execucao` e `indice`

MÉTODO	ROTA	FUNÇÃO
GET	`/opensearch.xml`	NOVO. Descrição OpenSearch para o navegador
GET	`/metrics`	NOVO. Métricas no formato texto do Prometheus
GET	`/api/info` · `/health`	Informações do servidor (versão 1.4.0) e sonda de vida

7.2 Exemplos

```
$ curl "http://localhost:8080/api/busca?q=%22inverted%20index%22&tamanho=5"

$ curl -X POST http://localhost:8080/api/fontes \
    -H "Content-Type: application/json" -H "X-Sonda-Key: <sua chave>" \
    -d '{"nome":"manuais","caminho":"/dados/manuais","incluir":["*.md","*.html"],"observar":true}'

$ curl -X POST "http://localhost:8080/api/fontes/manuais/indexar?modo=completa" \
    -H "X-Sonda-Key: <sua chave>"

$ curl -OJ "http://localhost:8080/api/busca/exportar?q=farol&formato=csv"

$ curl "http://localhost:8080/api/sugestoes?q=indice%20inv"
{ "consulta": "indice inv", "documentos": [ { "chave": "exemplos:tecnologia/indice-invertido.md",
  "titulo": "Índice invertido: a estrutura por trás de toda busca", "fonte": "exemplos",
  "caminho": "tecnologia/indice-invertido.md", "tipo": "md" } ] }
```

Erros seguem o padrão Problem Details (RFC 9457), com `title` e `detail` em português. Uma consulta com parêntese aberto, por exemplo, devolve 400 com a explicação do problema.

```
{ "title": "Consulta inválida.", "status": 400, "detail": "Faltou fechar um parêntese." }
```

CAPÍTULO 08

Integrações

A Sonda conversa com o navegador, leitores de feed, planilhas, ferramentas de monitoramento e outros sistemas usando só padrões abertos. Nada precisa ser instalado além do próprio servidor, e nada sai da máquina sem que você configure.

8.1 Navegador (OpenSearch)

O painel anuncia `/opensearch.xml` no cabeçalho da página. Com isso o navegador reconhece a Sonda como mecanismo de busca e passa a mostrar as sugestões de `/api/sugestoes?formato=opensearch` enquanto você digita na barra de endereços.

1. Abra o painel pelo menos uma vez no navegador.
2. **Chrome e Edge:** vá em Configurações → Mecanismo de pesquisa → Gerenciar mecanismos; a Sonda aparece em "Inativos" ou pode ser adicionada com a URL `http://localhost:8080/#/?q=%s`. Defina um atalho, como `so`.
3. **Firefox:** clique com o botão direito na barra de endereços e escolha **Adicionar "Sonda"**.
4. Na barra de endereços, digite o atalho, um espaço e a consulta.

8.2 Feed Atom

Qualquer busca vira um feed: `http://localhost:8080/api/busca/feed?q=farol&fonte=manuais`. Os filtros são os mesmos do painel (`tipo`, `idioma`, `fonte`, `ordem`). Num leitor de feeds (Thunderbird, NetNewsWire, Feedly com acesso à rede local), documentos novos ou alterados que casam com a busca aparecem sozinhos. Com `ordem=recentes`, o feed acompanha as últimas alterações de uma pasta.

8.3 Planilhas e dados

O botão **Exportar** da busca e a rota `/api/busca/exportar` entregam até 1.000 resultados em três formatos:

FORMATO	BOM PARA	DETALHES
`csv`	Excel, LibreOffice, Power BI	UTF-8 com BOM, separador vírgula. Células que começam com `=`, `+`, `-` ou `@` recebem um apóstrofo para não virarem fórmula
`json`	Scripts e relatórios	Envelope com `consulta`, `total`, `exportados`, `geradoEm` e `resultados`
`ndjson`	`jq`, Logstash, cargas em lote	Um resultado por linha, sem envelope

Colunas: `posicao`, `pontuacao`, `titulo`, `fonte`, `caminho`, `tipo`, `idioma`, `tamanho`, `modificado` e `trecho`. No Power BI ou no Excel, use *Obter dados → Da Web* com a URL da exportação em CSV para atualizar a planilha com um clique.

8.4 Monitoramento (Prometheus)

`/metrics` expõe o estado do servidor no formato texto do Prometheus, pronto para alertas e painéis no Grafana. Com `ProtectReads` ligado, a rota também exige `X-Sonda-Key`.

MÉTRICA	TIPO	SIGNIFICADO
`sonda_info{versao}`	gauge	Sempre 1; o rótulo traz a versão
`sonda_documentos`	gauge	Documentos pesquisáveis
`sonda_documentos_excluidos`	gauge	Excluídos aguardando mesclagem
`sonda_segmentos` · `sonda_indice_bytes`	gauge	Segmentos publicados e tamanho em disco
`sonda_buscas_total`	counter	Buscas atendidas desde o início do processo
`sonda_buscas_sem_resultado_total`	counter	Buscas que não acharam nada
`sonda_busca_duracao_ms_soma` · `_contagem`	counter	Para calcular a latência média
`sonda_execucoes_total{situacao}`	counter	Execuções de indexação no histórico, por situação
`sonda_fila_indexacao` · `sonda_indexando`	gauge	Fontes na fila e 1 enquanto uma indexação roda
`sonda_webhooks_enviados_total` · `_falhas_total`	counter	Webhooks entregues e que falharam

PROMETHEUS.YML

```
scrape_configs:
  - job_name: sonda
    metrics_path: /metrics
    scrape_interval: 30s
    static_configs:
      - targets: ["127.0.0.1:8080"]
    # só com ProtectReads ligado (Prometheus 2.55 ou mais novo)
    http_headers:
      X-Sonda-Key:
        values: ["<sua chave>"]
```

PROMQL

```
# latência média das buscas nos últimos 5 minutos, em ms
rate(sonda_busca_duracao_ms_soma[5m]) / rate(sonda_busca_duracao_ms_contagem[5m])

# proporção de buscas sem resultado
rate(sonda_buscas_sem_resultado_total[1h]) / rate(sonda_buscas_total[1h])
```

8.5 Webhooks

Webhooks avisam outros sistemas quando algo muda: uma indexação terminou (`execucao`) ou um novo ponto de gravação foi publicado (`indice`). Ficam desligados até você cadastrar pelo menos uma URL.

APPSETTINGS.JSON

```
"Sonda": {
  "Webhooks": {
    "Urls": [ "http://127.0.0.1:9000/sonda" ],
    "Secret": "troque-por-um-segredo-longo",
    "Events": [ "execucao", "indice" ],
    "TimeoutSeconds": 10
  }
}
```

Por variáveis de ambiente: `Sonda__Webhooks__Urls__0`, `Sonda__Webhooks__Secret` e `Sonda__Webhooks__Events__0`. Cada evento vira um POST com o corpo `{ "evento", "momento", "dados" }` e o cabeçalho `X-Sonda-Evento`. Com `Secret`, o servidor também envia `X-Sonda-Assinatura: sha256=<hex>`, o HMAC-SHA256 do corpo exato com o segredo. Quem recebe deve recalcular a assinatura e comparar em tempo constante antes de confiar no conteúdo. O envio acontece em segundo plano; falhas vão para o log e para a métrica `sonda_webhooks_falhas_total`.

PYTHON (FLASK)

```
import hashlib, hmac
from flask import Flask, abort, request

SEGREDO = b"troque-por-um-segredo-longo"
app = Flask(__name__)

@app.post("/sonda")
def receber():
    esperado = "sha256=" + hmac.new(SEGREDO, request.get_data(), hashlib.sha256).hexdigest()
    if not hmac.compare_digest(esperado, request.headers.get("X-Sonda-Assinatura", "")):
        abort(401)
    evento = request.get_json()
    print(evento["evento"], evento["dados"])
    return "", 204
```

NODE.JS

```
import http from 'node:http';
import { createHmac, timingSafeEqual } from 'node:crypto';

const segredo = process.env.SONDA_SEGREDO;

http.createServer((req, res) => {
  const partes = [];
  req.on('data', (parte) => partes.push(parte));
  req.on('end', () => {
    const corpo = Buffer.concat(partes);
    const esperado = Buffer.from('sha256=' + createHmac('sha256',
segredo).update(corpo).digest('hex'));
    const recebido = Buffer.from(req.headers['x-sonda-assinatura'] ?? '');
    if (esperado.length !== recebido.length || !timingSafeEqual(esperado, recebido)) {
      res.writeHead(401).end();
      return;
    }
    const { evento, dados } = JSON.parse(corpo);
    console.log(evento, dados);
    res.writeHead(204).end();
  });
}).listen(9000, '127.0.0.1');
```

8.6 Exemplos de código

A tela Integrações monta estes exemplos com a última busca feita. Todos leem a mesma resposta: `total`, `parsedQuery`, `elapsedMs` e a lista `hits`, em que cada item tem `key`, `title`, `path`, `source`, `extension`, `language`, `score` e `snippet`.

CURL

```
curl -s "http://localhost:8080/api/busca?q=farol&tamanho=5" | jq '.hits[] | {title, score}'
```

POWERSHELL

```
$r = Invoke-RestMethod "http://localhost:8080/api/busca?q=farol&tamanho=5"
$r.hits | Select-Object title, score, path
```

JAVASCRIPT

```
const resposta = await fetch('http://localhost:8080/api/busca?q=' + encodeURIComponent('farol'));
const { total, hits } = await resposta.json();
hits.forEach((h) => console.log(h.score.toFixed(3), h.title));
```

PYTHON

```
import requests

r = requests.get("http://localhost:8080/api/busca", params={"q": "farol", "tamanho": 5})
for hit in r.json()["hits"]:
    print(f"{hit['score']:.3f}  {hit['title']}")
```

Para rotas de escrita, acrescente o cabeçalho `X-Sonda-Key`. Para chamar a API de outra origem no navegador, libere a origem em `Sonda:Api:AllowedOrigins`.

CAPÍTULO 09

Configuração

As opções ficam em `src/Sonda.Server/appsettings.json` e podem ser sobrescritas por variáveis de ambiente com dois sublinhados, como `Sonda__Search__Similarity=TFIDF`.

9.1 Opções

OPÇÃO	PADRÃO	FUNÇÃO
`Urls`	`http://127.0.0.1:8080`	Endereço do servidor
`Sonda:IndexPath` · `DataPath`	`dados/indice` · `dados`	Pasta do índice e do catálogo
`Sonda:Analysis:Language`	`auto`	auto, pt, en ou nenhum (sem stemming)
`Sonda:Analysis:DefaultLanguage`	`pt`	Idioma quando a detecção não decide
`Sonda:Analysis:StopWords` · `FoldAccents`	`true` · `true`	Remoção de stopwords e de acentos
`Sonda:Indexing:RamBufferMb`	`128`	Memória do buffer antes de gravar um segmento
`Sonda:Indexing:Parallelism`	`0`	Threads de análise; 0 usa todos os núcleos
`Sonda:Indexing:MaxFileMb`	`32`	Arquivos maiores são ignorados e aparecem como falha
`Sonda:Indexing:StoreBody`	`true`	Guarda o texto para trechos, visualização, download e semelhantes
`Sonda:Indexing:MergeFactor` · `AutoMerge`	`10` · `true`	Quantos segmentos disparam uma mesclagem
`Sonda:Indexing:IndexOnStartup`	`true`	Indexação incremental de todas as fontes ao iniciar
`Sonda:Indexing:WatchDebounceSeconds`	`3`	Espera após a última alteração antes de reindexar
`Sonda:Search:Similarity`	`BM25`	BM25 ou TFIDF
`Sonda:Search:K1` · `B`	`1,2` · `0,75`	Parâmetros do BM25
`Sonda:Search:TitleBoost`	`2`	Peso do título em relação ao corpo
`Sonda:Search:DefaultOperator`	`And`	And ou Or
`Sonda:Search:MaxResultWindow` · `SnippetLength`	`1000` · `240`	Resultados navegáveis e tamanho do trecho
`Sonda:Api:ApiKey` · `ProtectReads`	`vazio` · `false`	Chave de acesso e proteção também das leituras (inclui `/metrics`)
`Sonda:Api:AllowedOrigins`	`vazio`	Origens liberadas para CORS
`Sonda:Webhooks:Urls`	`vazio`	URLs que recebem os eventos; vazio desliga os webhooks
`Sonda:Webhooks:Secret`	`vazio`	Segredo do HMAC-SHA256 em `X-Sonda-Assinatura`
`Sonda:Webhooks:Events`	`execucao, indice`	Eventos enviados
`Sonda:Webhooks:TimeoutSeconds`	`10`	Tempo máximo de cada envio (1 a 120)

OPÇÃO	PADRÃO	FUNÇÃO
`Sonda:AllowedRoots`	vazio	Se preenchido, novas fontes precisam estar dentro dessas pastas
`Sonda:Sources`	exemplos	Fontes fixas: Name, Path, Include, Exclude e Watch

9.2 Variáveis do Docker

VARIÁVEL	FUNÇÃO
`SONDA_PORT`	Porta no computador (padrão 8080)
`SONDA_API_KEY`	Chave de acesso; os scripts geram uma aleatória
`SONDA_DOCUMENTOS`	Pasta do computador montada como somente leitura e indexada
`SONDA_IDIOMA` · `SONDA_RAM_MB`	Idioma da análise e memória do buffer

Qualquer outra opção pode ser passada ao contêiner no formato de variável de ambiente, por exemplo `Sonda__Webhooks__Urls__0=http://host.docker.internal:9000/sonda` no `docker-compose.yml`.

CAPÍTULO 10

Desempenho e ajuste

Números do comando `sonda benchmark` numa máquina com 2 vCPUs Xeon a 2,1 GHz, com 100 mil documentos sintéticos de 80 a 600 palavras e 1.000 consultas de 1 a 3 termos.

10.1 Resultados

MEDIDA	RESULTADO
Indexação	18,8 s · **5.333 documentos/s** · 16,3 MB/s de texto
Mesclagem para 1 segmento	9,6 s
Tamanho do índice	176,9 MB, com o texto completo armazenado
OU exato com contagem e facetas	p50 2,98 ms · p95 6,36 ms · 30.809 documentos avaliados
OU top 10 com WAND	p50 2,24 ms · p95 3,61 ms · **756 documentos avaliados**
E com saltos	p50 2,17 ms · p95 4,45 ms
E com trechos destacados	p50 2,99 ms · p95 4,78 ms

Exportações usam a mesma busca, sem facetas, e só acrescentam a formatação; sugestões pedem até 10 documentos sem contagem exata, então custam menos que uma busca comum. O painel **Consultas** e a métrica `sonda_busca_duracao_ms` mostram a latência real da sua máquina.

10.2 Como ajustar

- **Mais núcleos:** a indexação escala com `Parallelism`; o gargalo passa a ser o disco.
- **RamBufferMb maior** gera menos segmentos e menos mesclagens. Com pouca memória, use 32 ou 64.
- **StoreBody=false** reduz o índice pela metade, mas desliga trechos, visualização, download e semelhantes.
- **Otimizar** depois de uma carga grande deixa um segmento só e acelera as buscas.
- **Idioma fixo** (pt ou en) evita as duas variantes por palavra e deixa as consultas um pouco mais rápidas.
- Coloque a pasta `dados/` num SSD.

CAPÍTULO 11

Testes, CI e automação

O projeto tem casos de teste xUnit, uma integração contínua e uma automação que grava atividade diária no repositório.

11.1 Testes

ARQUIVO	O QUE PROVA
`AnalysisTests`	Tokenização, remoção de acentos, Porter com os exemplos do artigo original, famílias de palavras no RSLP, exceções, detecção de idioma e extração de HTML e Markdown
`StorageTests`	CRC32 de referência, inteiros variáveis, blocos FOR de 0 a 31 bits, normas monotônicas, mapa de bits, detecção de byte corrompido e trava de escrita
`SearchTests`	Pontuação BM25 igual à fórmula, peso do título, frases, prefixo, busca aproximada, exclusão, filtros, facetas, ampliação automática, ordem por data, trechos, explicação e TF-IDF
`IndexLifecycleTests`	Visibilidade só depois do commit, reabertura, atualização, exclusão, mesclagem com excluídos, listas longas com saltos e limpeza de arquivos antigos
`ConcurrencyTests`	6.000 documentos em 8 threads com vários flushes, buscas durante a indexação e WAND igual à busca exaustiva
`QueryParserTests`	Interpretação de cada operador, erros de sintaxe e distância de edição
`PipelineTests`	Indexação incremental de pastas reais: novos, alterados, removidos, completa e padrões de inclusão e exclusão

```
$ dotnet test tests/Sonda.Tests
```

11.2 Integração contínua

O arquivo `.github/workflows/ci.yml` roda a cada push na branch `main`: restaura, compila em Release com avisos tratados como erro, roda os testes, indexa os exemplos pela linha de comando, verifica os checksums, roda um benchmark curto e gera a imagem Docker.

11.3 Atividade diária

O arquivo `.github/workflows/diario.yml` grava dois commits por dia em `.github/atividade.txt`. Cada commit tem um horário principal e um de reserva, que só grava se o principal não tiver rodado:

REGISTRO	PRINCIPAL	RESERVA	CRON (UTC)
1º registro	00:19 (Brasília)	00:53	`19 3 * * *` e `53 3 * * *`
2º registro	03:23 (Brasília)	03:51	`23 6 * * *` e `51 6 * * *`

Os horários ficam na madrugada, fora do pico do GitHub Actions, em minutos quebrados (o começo de cada hora é o momento mais congestionado) e em faixas que nenhuma outra automação dos repositórios do autor usa, com pelo menos 30 minutos de distância de todas elas.

1. No GitHub, abra o repositório e vá em **Settings → Secrets and variables → Actions**.
2. Crie o secret `GIT_NAME` com o seu nome de usuário do GitHub.
3. Crie o secret `GIT_EMAIL` com o e-mail noreply do GitHub (`número+usuario@users.noreply.github.com`), para não expor o seu e-mail pessoal.
4. Em **Settings → Actions → General → Workflow permissions**, marque **Read and write permissions**.
5. Na aba **Actions**, abra **Atividade diária** e use **Run workflow** para testar.

CAPÍTULO 12

Segurança e privacidade

A Sonda foi feita para rodar na sua máquina ou na sua rede, sem depender de nenhum serviço de fora.

- O repositório não traz senhas, chaves, endereços nem dados pessoais. O `.env` é gerado na sua máquina e está no `.gitignore`, assim como a pasta `dados/`.
- Os documentos de exemplo foram escritos para o projeto; os manuais e a política de senhas são de uma empresa fictícia.
- O servidor escuta em `127.0.0.1` por padrão, e o Docker prende a porta ao mesmo endereço.
- Chave de acesso comparada em tempo constante (`CryptographicOperations.FixedTimeEquals`); no painel ela fica só na memória da aba.
- `Sonda:AllowedRoots` impede que alguém com a chave cadastre pastas fora das raízes permitidas.
- Cabeçalhos de segurança em todas as respostas: `Content-Security-Policy` sem scripts externos nem inline, `X-Frame-Options`, `nosniff` e `Referrer-Policy`.
- O painel monta o HTML com `textContent` e elementos criados um a um, então trechos de documentos nunca são interpretados como HTML. O feed Atom e a descrição OpenSearch escapam todo o texto como XML.
- Exportações em CSV neutralizam células que começariam uma fórmula (`=`, `+`, `-`, `@`), evitando injeção de fórmulas no Excel e no LibreOffice.
- Webhooks só saem para as URLs que você configurou e podem ser assinados com HMAC-SHA256, para que quem recebe confirme a origem e a integridade de cada evento.
- Histórico e buscas salvas ficam só no navegador de quem pesquisou; as estatísticas de busca do servidor ficam só na memória e somem ao reiniciar.
- Scripts, estilos e comentários das páginas HTML indexadas são descartados na extração.
- Nada é enviado para fora por padrão: sem telemetria, sem CDN, sem chamadas externas.

CAPÍTULO 13

Solução de problemas

Os sintomas mais comuns, a causa provável e o que fazer.

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
“O índice já está aberto para gravação por outro processo”	O servidor está rodando e a linha de comando tentou indexar	Pare o servidor ou use o painel. Comandos de leitura funcionam normalmente
A barra de estado mostra “só leitura”	Outro processo segura o `write.lock`	Feche o outro processo e reinicie o servidor
Nenhum resultado para uma palavra que existe	Ela é stopword ou o idioma detectado gerou outro radical	Use o **Analisador** na tela Termos para ver como a palavra é indexada
Uma fonte não indexa	A pasta não existe no servidor ou está fora de `AllowedRoots`	Confira o caminho na tela Fontes. No Docker, use o caminho de dentro do contêiner
Arquivos aparecem como falha	Maiores que `MaxFileMb` ou sem permissão de leitura	Aumente o limite ou ajuste as permissões
Acentos estranhos nos resultados	Arquivo em codificação diferente de UTF-8 e Latin-1	Converta o arquivo para UTF-8
“Checksum não confere” no verificar	Arquivo corrompido no disco ou cópia incompleta	Restaure o backup ou apague `dados/indice` e reindexe
Alterações na pasta não aparecem	Observar alterações desligado ou rede que não emite eventos	Clique em **Indexar** na fonte ou ligue **Observar**
Porta 8080 ocupada	Outro serviço na mesma porta	Mude `SONDA_PORT` no `.env` ou `Urls` no appsettings
O indicador “ao vivo” fica apagado	Um proxy está segurando a conexão de eventos	Desligue o buffer do proxy para `/api/eventos`. A busca continua funcionando
A Sonda não aparece como mecanismo de busca no navegador	O navegador só lê o OpenSearch depois de visitar a página	Abra o painel uma vez ou cadastre manualmente `/#/?q=%s`
Webhook não chega	URL errada, receptor fora do ar ou fora do alcance do contêiner	Veja o log e `sonda_webhooks_falhas_total`; no Docker, use `host.docker.internal`
Receptor rejeita a assinatura	O corpo foi relido ou alterado antes do HMAC, ou o segredo é outro	Calcule o HMAC sobre os bytes crus recebidos, com o mesmo `Secret`
`/metrics` responde 401	`ProtectReads` ligado	Envie `X-Sonda-Key` no scrape (`http_headers`)
CSV abre com acentos quebrados	Planilha importou sem detectar UTF-8	Abra o arquivo direto (ele tem BOM) ou escolha UTF-8 na importação

CAPÍTULO 14

Novidades da 1.5.0

A 1.5.0 ajuda quem digita errado, mostra por que os modelos de relevância discordam, guarda documentos importantes e protege o índice com cópias de segurança verificáveis. A paleta, as fontes e o formato binário do índice não mudaram: um índice criado na 1.4.0 abre na 1.5.0 sem reindexar.

14.1 Você quis dizer?

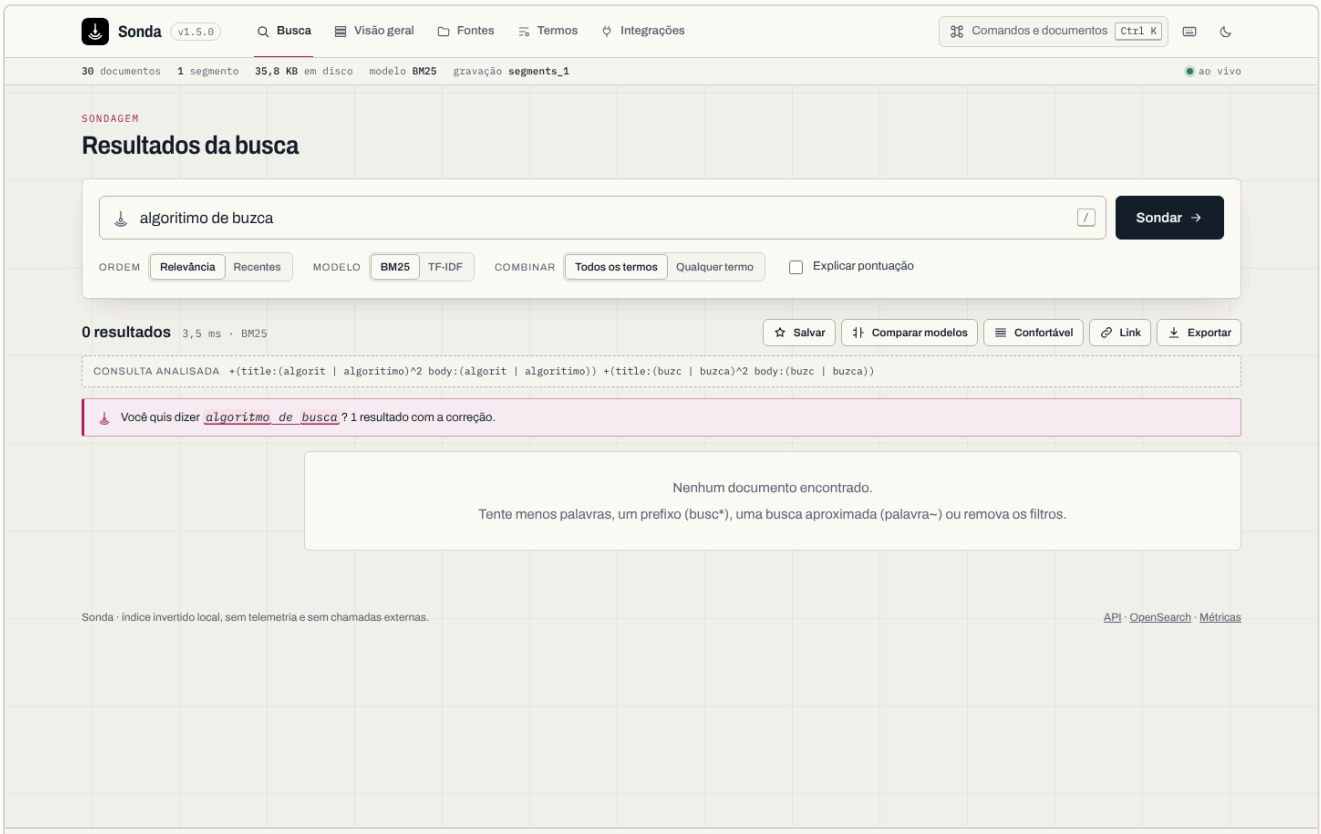
Quando uma palavra da consulta não existe em nenhum documento, a Sonda procura no dicionário do índice o termo mais próximo e mostra a consulta corrigida acima dos resultados, com a quantidade de documentos que ela encontra. Um clique refaz a busca.

01 Cada palavra passa pela mesma análise da busca (minúsculas, acentos e radical RSLP ou Porter). Se algum radical tiver documentos, a palavra fica como está.

02 Para as desconhecidas, o dicionário do campo `corpo` é percorrido a partir da primeira letra e cada termo é comparado pela distância de Damerau-Levenshtein: até 1 edição em palavras curtas e até 2 a partir de 6 letras. Ganha a menor distância e, no empate, o termo presente em mais documentos.

03 O radical escolhido volta a ser palavra: a Sonda relê o texto armazenado de até 3 documentos que o contêm e usa a grafia original mais parecida com o que foi digitado, com acentos (`navegasao` vira `navegação`).

04 Operadores, campos, filtros, prefixos (`busc*`), buscas aproximadas (`termo~`) e exclusões (`-termo`) são preservados. A sugestão só aparece se a consulta corrigida encontrar documentos.

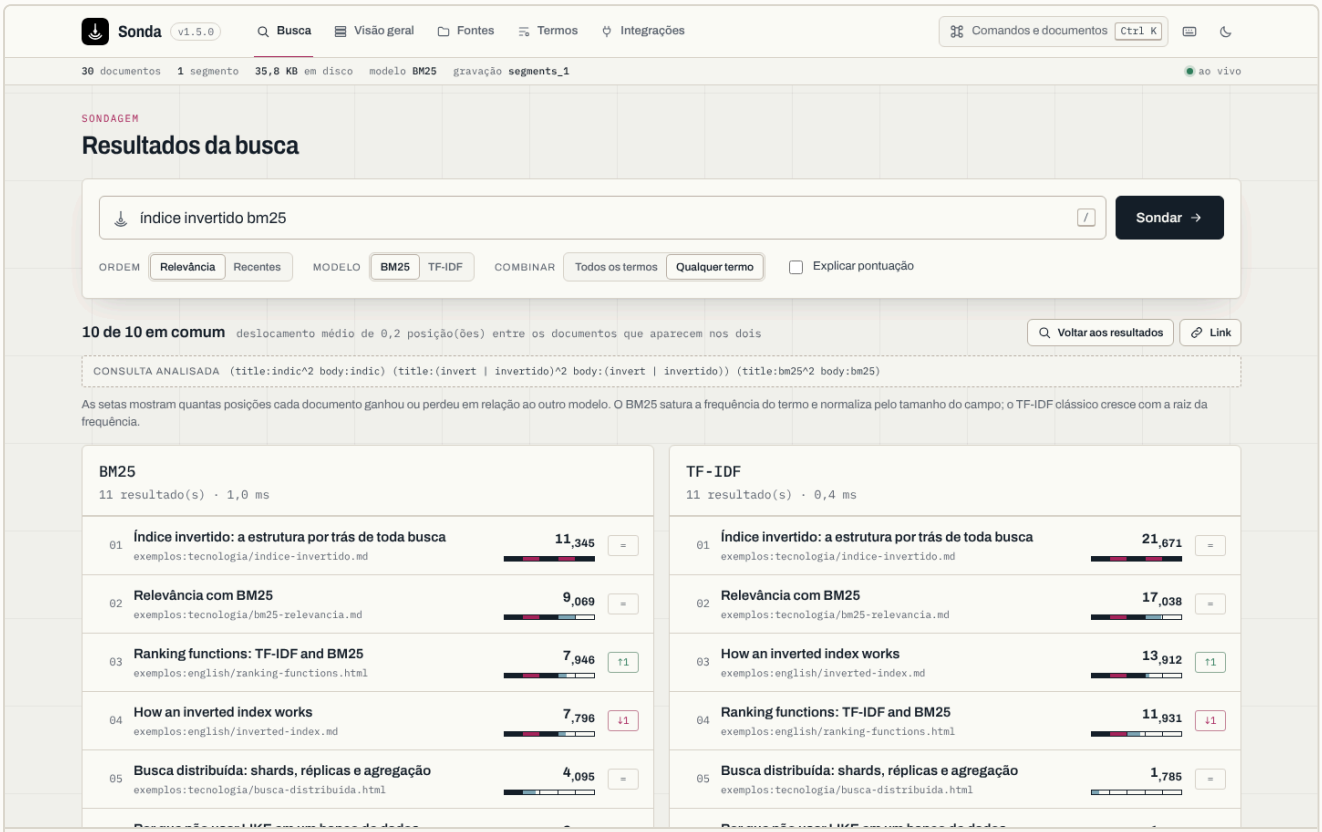


algoritimo de buzca · a sugestão destaca as palavras corrigidas

A sugestão vem no campo `suggestion` de `GET /api/busca` (só na primeira página), em `GET /api/ortografia?q=` e no comando `sonda corrigir "consulta"`. O `sonda buscar` também avisa quando há correção.

14.2 Comparar BM25 e TF-IDF

O botão **Comparar modelos** nos resultados (ou a paleta de comandos, em *Esta busca*) mostra o top 10 dos dois modelos lado a lado para a mesma consulta e os mesmos filtros. No topo aparecem quantos documentos os dois têm em comum e o deslocamento médio de posição; em cada linha, uma etiqueta indica quantas posições o documento subiu (`↑`), desceu (`↓`), se manteve (`=`) ou se só aparece num dos modelos (`novo`). Clicar num título abre o documento.



índice invertido bm25 · qualquer termo · 10 de 10 em comum

A mesma comparação sai em `GET /api/busca/comparar?q=&top=&operador=&tipo=&idioma=&fonte=` e em `sonda comparar "consulta" --top 10`, com `--json` para scripts. O link da tela guarda o parâmetro `comparar=1`.

14.3 Documentos marcados

Cada resultado ganhou um marcador ao lado do título, e o documento aberto tem o botão **Marcar** e o atalho `m`. Os marcados aparecem na tela inicial da busca, em *Documentos marcados*, e na paleta de comandos (`Ctrl K`). Como as buscas salvas, ficam só neste navegador (até 50), guardando chave, título, fonte e caminho; nada vai para o servidor.

ATALHO	AÇÃO
`m`	Marcar ou desmarcar o documento aberto
`Ctrl K`	Paleta: documentos marcados, comparar modelos e baixar cópia de segurança

14.4 Cópia de segurança e restauração

Uma cópia de segurança é um `.zip` com exatamente os arquivos do último commit (o `segments_N` e os arquivos dos segmentos que ele lista, inclusive os mapas de exclusão) e um manifesto `sonda-backup.json`. Como os segmentos são imutáveis, a cópia é consistente mesmo com o servidor aberto e indexando.

01 **Pelo painel:** Visão geral › Manutenção › **Cópia de segurança**. Com `Sonda:Api:ApiKey` definido, a rota exige a chave mesmo quando as leituras são públicas.

02 **Pela API:** `GET /api/indice/backup` devolve `sonda-indice-<geração>-<data>.zip`. O arquivo é montado num temporário apagado ao final do envio.

03 **Pela linha de comando:** `sonda backup copia.zip`, também com o servidor aberto.

04 **Para restaurar:** pare o servidor e rode `sonda restaurar copia.zip --substituir`. A cópia é extraída em `indice.restaurando-<data>`, o tamanho de cada arquivo é conferido com o manifesto, o índice é aberto e o CRC32 de todos os arquivos é verificado. Só então o índice atual é guardado como `indice.anterior-<data>` e a cópia entra no lugar. Se algo falhar, nada é alterado.

> A restauração recusa entradas com caminho (`../`, barras), nomes fora do padrão do índice e arquivos que não estão no manifesto. Sem `--substituir`, ela só grava num índice vazio ou inexistente.

14.5 Novas rotas e comandos

ROTA OU COMANDO	FUNÇÃO
`GET /api/busca/comparar`	Top BM25 e TF-IDF com posições, documentos em comum e deslocamento médio
`GET /api/ortografia?q=`	Consulta corrigida, correções (original, sugestão, distância, documentos) e total
`GET /api/indice/backup`	Cópia de segurança do último commit em .zip (sempre exige a chave, se houver)
`sonda comparar "consulta"`	Comparação no terminal. `--top`, `--ou`, `--tipo`, `--fonte`, `--json`
`sonda corrigir "consulta"`	Mostra a correção sugerida. `--json`
`sonda backup [arquivo.zip]`	Grava a cópia de segurança
`sonda restaurar <arquivo.zip>`	Restaura conferindo checksums. `--substituir`

14.6 Testes

São 103 casos. Os novos provam que a correção encontra a palavra certa e devolve a grafia com acentos, que não sugere nada quando todas as palavras existem, que preserva operadores, campos e filtros, que a comparação é simétrica entre os modelos e que uma cópia restaurada devolve os mesmos resultados, passa na verificação de integridade e recusa arquivos fora do padrão.